Modular Chess AI Engine

A Scalable, Strategy-Pattern-Driven Chess Platform

Academic Year: 2025-2026
Technology Stack: React.js, JavaScript (ES6+)
Architecture: Layered Architecture, SOLID Principles, Strategy Design Pattern
Live URL: https://modularchess.vercel.app/
Report Date: April 2026

---

1. Executive Summary

The Modular Chess AI Engine is a full-featured browser-based chess platform developed using React.js and JavaScript. Instead of building a basic chess game, this project was designed as a flexible AI strategy framework where multiple artificial intelligence approaches can be tested, compared, and extended inside a clean and scalable architecture.

The platform supports Human vs AI gameplay, AI vs AI simulation, automated tournaments, ELO-based ranking systems, configurable time controls, and full game replay functionality. Every module follows SOLID principles and uses the Strategy Design Pattern to ensure scalability and maintainability.

This project demonstrates strong object-oriented design, design patterns, and software engineering principles while solving a real-world AI problem.

---

2. Project Overview

2.1 Motivation and Objectives

Chess has always been one of the most important benchmark problems in Artificial Intelligence. The goal of this project was not just to create a playable chess game, but to build a system where intelligent algorithms are modular, swappable, and easy to compare.

Main Objectives

- Design a maintainable system using SOLID principles
- Implement multiple AI strategies using the Strategy Pattern
- Support AI vs AI tournament simulation
- Track AI performance using ELO Rating System
- Create a UML-documented architecture for academic evaluation
- Deploy a complete working web application

2.2 Scope of the Project

The system includes the following modules:

Chess Gameplay

- Full chess rules implementation
- Move validation
- Check, checkmate, stalemate detection
- Turn-based game engine

AI Strategies

- Minimax Algorithm
- Alpha-Beta Pruning
- Heuristic Evaluation Strategy

Tournament System

- Multi-round AI vs AI tournaments
- Match statistics
- Ranking leaderboard

ELO Rating System

- Dynamic rating updates after every match

Time Control

- Blitz Mode (5 minutes)
- Classical Mode (90 minutes)
- Unlimited Mode

Logging and Replay

- Full move history recording
- Board reconstruction
- Replay functionality

User Interface

- Interactive React chessboard
- Tournament dashboard
- Replay system
- Control panel

---

3. Technology Stack

Layer	Technology	Purpose
Frontend UI	React.js	Interactive user interface
Game Logic	JavaScript ES6+	Chess rules and state management
AI Engine	Vanilla JavaScript	AI strategy implementation
Build Tool	Vite	Fast development and production build
Deployment	Vercel	Live deployment
Version Control	Git + GitHub	Source code management

---

4. System Architecture

4.1 Architectural Philosophy

The system follows a layered architecture where every layer has a single responsibility and communicates only through clearly defined abstractions.

This improves:

- Loose coupling
- High cohesion
- Maintainability
- Scalability
- Testability

Core Principles Used

Single Responsibility Principle (SRP)

Every class performs only one task.

Open Closed Principle (OCP)

New AI strategies can be added without changing existing code.

Dependency Inversion Principle (DIP)

High-level modules depend on interfaces, not concrete implementations.

4.2 Main Layers

UI Layer

- ChessBoard
- ControlPanel
- TournamentPanel
- ReplayPanel

Game Engine Layer

- GameEngine.js

Agent Layer

- Agent
- HumanAgent
- AIAgent

Game Rules Layer

- GameRules
- GameState
- Board
- Piece

Strategy Layer

- Strategy Interface
- MinimaxStrategy
- AlphaBetaStrategy
- HeuristicStrategy

Tournament Layer

- TournamentManager
- RatingSystem
- EloRatingSystem

Time Policy Layer

- TimePolicy
- BlitzPolicy
- ClassicalPolicy
- UnlimitedPolicy

Logging Layer

- MoveLogger
- ReplaySystem

5. Design Patterns Used

5.1 Strategy Pattern (Main Pattern)

This is the most important design pattern used in the project.

It allows multiple AI algorithms to be used interchangeably.

Example

Instead of writing AI logic directly inside AIAgent, the system injects a strategy object.

This allows:

- Runtime strategy switching
- Easy testing of multiple AI models
- No modification of GameEngine when adding new strategies

Implemented Strategies

- Minimax
- Alpha-Bta Pruning
- Heuristic Strategy

This follows the Open Closed Principle perfectly.

5.2 Observer Pattern

GameEngine acts as the Subject.

Observers include:

- MoveLogger
- UI Components
- TournamentManager

Whenever a move happens, all observers are notified automatically.

This reduces dependency between modules.

5.3 Template Method Pattern

Used in:

RatingSystem → EloRatingSystem

The parent class defines the structure of rating updates while the child class implements the exact ELO logic.

5.4 Factory Pattern

Used in:

StrategyFactory

This creates strategy objects like:

- Minimax
- Alpha-Beta
- Heuristic

This centralizes object creation and improves maintainability.

---

6. AI Strategy Implementation

6.1 Heuristic Strategy

This strategy checks all legal moves and chooses the move with the best immediate board position.

It does not perform deep search.

Evaluation Uses

- Material value of pieces
- Piece-square tables
- Center control
- King safety
- Piece activity

Advantage

Very fast

Disadvantage

Weak long-term planning

6.2 Minimax Strategy

Minimax is a recursive search algorithm where:

- AI tries to maximize score
- Opponent tries to minimize score

It searches all possible moves up to a fixed depth.

Time Complexity

O(b^d)

Where:

- b = branching factor
- d = search depth

Advantage

Correct strategic decisions

Disadvantage

Very slow for deeper levels

6.3 Alpha-Beta Pruning

This improves Minimax by removing unnecessary branches.

It uses:

- Alpha = best score for maximizer
- Beta = best score for minimzer

When:

Alpha ≥ Beta

the branch is pruned.

Advantage

Much faster than Minimax

Best choice for competitive AI

---

7. ELO Rating System

The project uses the ELO rating system to rank AI agents based on performance.

Formula

Expected Score

E(A) = 1 / (1 + 10^((RB - RA)/400))

Rating Update

R(new) = R(old) + K(S - E)

Where:

- R = rating
- K = K-factor (32)
- S = actual result
- E = expected result

Example

If:

- Agent A = 1500
- Agent B = 1300

and Agent A wins,

Agent A gains rating points and Agent B loses rating points.

This creates a fair ranking system.

---

8. UML Diagrams

The project includes complete UML documentation.

Class Diagram

Shows:

- Classes
- Methods
- Fields
- Inheritance
- Composition
- Dependencies

ER Diagram

Shows:

- AGENT
- GAME
- TOURNAMENT
- MOVE
- GAME_STATE
- RATING_HISTORY

and their relationships.

Sequence Diagram

Shows the Human vs AI move flow:

1. User makes move
2. GameEngine validates
3. MoveLogger records
4. AI selects move
5. Strategy computes best move
6. Game updates
7. UI renders board

Use Case Diagram

Actors:

- Human Player
- AI Agent
- System Admin

Main Use Cases:

- Start Game
- Make Move
- Run Tournament
- View Rankings
- Replay Game

---

9. SOLID Principles Used

Principle	Application
Single Responsibility	MoveLogger only logs, GameRules only validates
Open Closed	New strategy added without modifying old code
Liskov Substitution	All strategies work through Strategy interface
Interface Segregation	Small focused interfaces
Dependency Inversion	AIAgent depends on Strategy interface

---

10. Key Features

Human vs AI Gameplay

- Interactive chessboard
- Move highlighting
- Check and checkmate detection
- Castling
- En passant
- Pawn promotion

AI vs AI Simulation

- Strategy comparison
- Search depth configuration
- Think-time analysis

Tournament System

- Round-robin tournaments
- Live rankings
- ELO updates
- Match history

Replay System

- Step-by-step replay
- Move history
- Export game logs

---

11. Challenges and Solutions

Challenge	Solution
Decoupling AI from game logic	Strategy Pattern
Preventing illegal moves	Board cloning + validation
Slow Minimax search	Alpha-Beta pruning
State immutability	Deep cloning
UI synchronization	Async Promise handling
Correct ELO updates	Centralized EloRatingSystem

---

12. Future Enhancements

Possible future upgrades:

- Iterative Deepening Search
- Opening Book
- Neural Network Evaluation
- REST API Backend
- Parallel Search using Web Workers

- Multiplayer Support using WebSockets
- Puzzle Mode

The architecture already supports these extensions without major rewriting.

---

13. Conclusion

The Modular Chess AI Engine proves that strong software architecture and AI logic can work together effectively.

By using:

- Strategy Pattern
- SOLID Principles
- Layered Architecture
- Design Patterns
- UML Documentation

the system becomes much more than a simple chess game.

It becomes a scalable AI framework where intelligence is modular and future improvements are easy to implement.

This project demonstrates both technical depth and software engineering maturity, making it suitable for academic submission as well as professional portfolio presentation.